Bug 1 SQL Search and Status filter

Location
backend/src/main/java/
TaskRepository.java

How I found the bug
I tested the task search API using both a search keyword and a status filter. Some result did not reflect the expected status or archived-task filtering.

Root Cause
The SQL query used AND and OR without grouping the search condition
The original logic was effectively interpreted as:
(archived = false AND title matches) OR (description matches AND status matches)
Because SQL evaluates AND before OR, the filters could be bypassed in some cases.

Fix
I grouped the title and description search conditions:
archived = false AND (title matches OR description matches) AND status condition

Why I chose this fix
It is a small and focused change. It keeps the existing query structure while making the intended filtering logic explicit and correct

Location
TaskController.java

How I found the bug
I noticed that every search request calculated a complexity score and then called Thread.sleep

Root Cause

The Controller intentionally delayed the request based on the query length. This blocks the request thread and make the API unnecessarily slow.

Fix

I removed the artifical delay and the unnecessary complexity calculation.

Why I chose this fix

The delay does not Provide useful application functionality. Removing it improves API responsiveness without changing the actual task-search behavior

Bug 3: Invalid Pagination and Status Input

Location

TaskController.Java

How I found the bug

I checked boundary values such as Page 0, negative Page numbers, zero or negative Page Size, very large Page Size and invalid Status values.

Root Cause

The API did not Properly validate these request Parameters before Processing them.

Fix

I added validation for Pagination Parameters and restricted the Page Size to a reasonable maximum I also handled invalid Status value as client input errors.

Bug 4 - In Memory Pagination

Location

Task Controller. Java

observation

The repository currently retrieves all matching tasks and Pagination is then Performed using Java subList()

Why I did not change it

Moving Pagination completely into the database would require a larger change to the repository and response/count logic.

Because this exercise has a limited timebox, I kept the existing structure and focused on the higher-impact correctness issues.

Remaining Risk

If the number of tasks become very large, loading all matching rows before Pagination can become inefficient.

Future Improvement

I would use database-level Pagination with Spring Data Pageable and return only the required records.

Bug 5 - Fronted Pagination After Filtering

LoCation

Fronted task list / search logic

How I found the issue

If the user is on a later Page and then change the search or status filter, the current Page may no longer be valid for the new filtered result set.

Fix
The Page is reset to Page 1 whenever the search query or status filter change

why I chose this fix
A new filter represent a new result, so starting from the first page give Predictable behavior and avoids showing an empty page unnecessarily.

Bug 6 Multiple API Request while Typing

Location
Fronted Search logic

How I found the issue
Typing a search term can trigger a request for every character entered.

Root Cause.
There was no debounce around the search input.

Fix
I added a debounce so the API request is made only after the user stops typing for a short period.

Why I chose this fix
It reduce unnecessary API request and Prevents multiple overlapping request while keeping the search experience responsive.

Bug 7 Fronted Loading / Error Handling

Location
Fronted API/task loading logic

How I found the issue
The loading state was not guaranteed to be reset when an API request failed.